

Arbres Lexicogràfics Eficients: Informe

GRAU A – Q1 CURS 2025-2026

Departament de Ciències de la Computació
Universitat Politècnica de Catalunya

LAURA MORENO VALENCIA

`laura.christel.moreno@estudiantat.upc.edu`

VÍCTOR HUERTES MONTES

`victor.huertes@estudiantat.upc.edu`

ABRAHAM RUIZ VASQUEZ

`abraham.ruiz.vasquez@estudiantat.upc.edu`

POL RIVEIRO CLARÀ

`pol.riveiro@estudiantat.upc.edu`

Resum

Aquest projecte explora el disseny, implementació i evaluació experimental de les estructures d'arbres lexicogràfics per emmagatzematge i cerca de cadenes de text. Específicament, comparem el Naive Trie i el Radix Tree, analitzant les seves diferències en ús de memòria, compactesa i rendiment de cerca. L'objectiu principal és entendre com la compressió de prefixos comuns en arbres lexicogràfics pot millorar l'eficiència espacial i temporal en les aplicacions de processat de text.

Per aconseguir això, les dues estructures de dades han estat implementades en C++ i testejades en múltiples datasets amb característiques variades. Els experiments han mesurat ús total de memòria, nombre de nodes, distribució de profunditat i temps mig de cerca. Els resultats confirmen les expectacions teòriques: el Radix Tree aconsegueix una reducció de memòria de més del 90% en alguns casos comparat amb el Naive Tree, mantenint temps d'inserció i cerca competitius. Tot i això, el rendiment varia depenent de la redundància dels prefixos dintre de cada dataset.

Part I. Informe Tècnic

1 Introducció i Objectius

Un trie és una estructura de dades en forma d'arbre que utilitza l'indexatge de paraules per organitzar informació. Originalment, els tries van ser pensats per recollir una sèrie de strings dintre d'un alfabet fixat, però a la computació moderna són àmpliament utilitzades per eines de predicció i cerca a diversos tipus de dades. L'objectiu d'aquest treball és comprendre el funcionament d'aquestes estructures, implementant diferents optimitzacions de les mateixes i avaluant-les experimentalment.

2 Antecedents

L'estructura de dades trie (també coneguda com a arbre de prefixos) és un arbre ordenat utilitzat per representar una sèrie de paraules (strings) sobre un alfabet finit. Permet que els prefixos comuns facin servir els mateixos nodes, i emmagatzema només els caràcters que difereixen. Cada node en una trie està associat a un caràcter de l'alfabet, menys el node arrel que és un node buit, i el nombre màxim de fills que pot tenir un determinat node és igual al nombre de caràcters existents en l'alfabet. Pel propòsit d'aquest treball, es farà servir el diccionari ASCII com a referència, de manera que tots els fills quedaran ordenats alfabeticament seguint aquest estandard. El final d'una paraula quedarà representat per un valor associat al node, on s'indicarà l'index d'aquesta paraula dintre del conjunt del text que es faci servir com a dataset. És important recalcar que, per tal que una paraula aparegui al diccionari constituït per un trie, no cal que sigui un node fulla, ja que es pot fer una indexació de qualsevol node intermig que formi part d'una altra paraula més llarga.

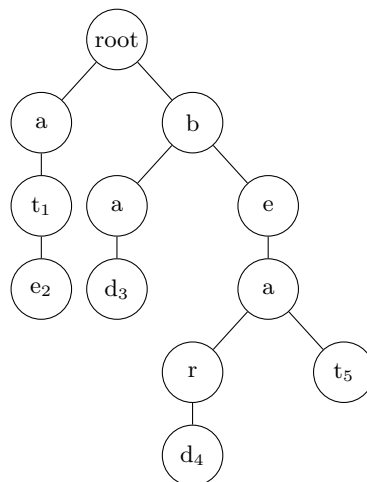


Figura 1: Exemple d'un trie que conté les paraules “at”, “ate”, “bad”, “bed”, “beard” i “beat”.

Tot i que, en general, les tries són estructures eficients per operacions de cerca de paraules o prefixos, aquesta eficiència normalment significa una penalització en l'ús de memòria. A una implementació bàsica d'un trie, cada node existent tindrà una quantitat de fills igual a la quantitat de caràcters existents en l'alfabet amb el que estem treballant. Això vol dir que en el nostre cas, per exemple, cada node té 128 nodes fills. Notem que a la Figura 1, per simplificació, omitem els fills conformatos per nodes buits. Per tant, la complexitat espacial d'un trie d'aquest tipus serà de l'ordre de $O(NM)$, on N representa el nombre de paraules i M la mida màxima d'una paraula.

L'eficiència en termes de memòria ha estat el motor pel desenvolupament de diverses variants de les tries tradicionals. En aquest treball discutirem dues de les principals alternatives: els Ternary Search Trees (TST) i els Radix Trees.

2.1 Ternary Search Trees

Un Ternary Search Tree (o arbre ternari de cerca) és un tipus de tree que busca aprofitar els avantatges espacials dels Binary Search Trees, reduint el numero de fills possibles per un node qualsevol. El seu funcionament és simple: cada node tindrà 3 possibles fills. El fill esquerre (*left child*), el fill dret (*right*

child) i el fill central (*middle child*). El valor del fill esquerre haurà de ser un valor menor al del node pare, el del fill dret haurà de ser major, y el fill central serà un punter al valor del següent símbol de la string a qui pertany el node central. Notem que el rendiment d'aquesta estructura depen directament de la profunditat de l'arbre.

En general, l'ús d'un TST sol ser el mateix que el d'un Trie estandard, però obtindrem el pitjor rendiment a un TST treballant amb cadenes ordenades, ja que la millor manera que tindrem de garantir un bon rendiment sera tenint un arbre balancejat.

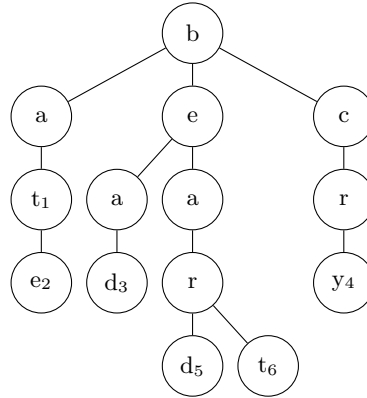


Figura 2: Exemple d'un TST que conté les paraules “at”, “ate”, “bad”, “cry”, “beard” i “beat”.

2.2 Radix Trees

Un Radix Tree és una altra variant de trie bàsic que busca estalviar espai en la memòria. L'idea principal d'aquesta estructura és comprimir els nodes que tenen un sol fill per així aconseguir reduir la profunditat de l'arbre i, per tant, el nombre de nodes necessaris per representar un conjunt de paraules. El seu funcionament consisteix en que cada aresta de l'arbre pot representar una cadena de caràcters en lloc d'un sol caràcter. Això significa que els nodes es poden fusionar entre si per només haver de tenir un que representa la cadena completa. Per exemple, representant el mateix conjunt de paraules {“at”, “ate”, “bad”, “bed”, “beard”, “beat”} amb un Radix Tree, tindriem la següent estructura:

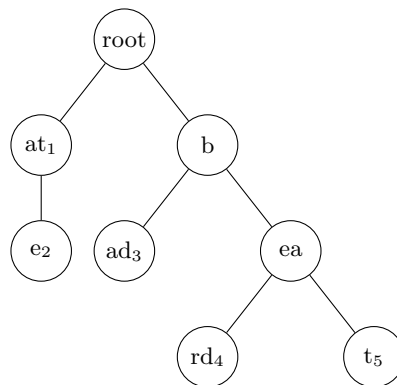


Figura 3: Exemple d'un Radix tree que conté les paraules “at”, “ate”, “bad”, “bed”, “beard” i “beat”.

Com veurem més endavant, aquesta estructura millora significativament l'ús de memòria en comparació amb un tree naive i ternari en la majoria de casos (incloent en molt dels textos d'exemples proporcionats), però en cas pitjor tindrà un rendiment similar.

2.3 Patricia Trees

Un Patricia Tree (Practical Algorithm To Retrieve Information Coded In Alphanumeric) és una variant comprimida de Radix Tree on els nodes element es poden fusionar amb els seus pares, mantenint només aquells necessaris per diferenciar entre les paraules del conjunt. El Patricia té la característica de que en comptes de guardar i comparar cadenes de caràcters, ho fa a nivell de bits.

En aquest treball només els tindrem en compte a nivell teòric, ja que considerem que les altres estructures anteriorment descrites seran més senzilles d'implementar i manipular amb l'objectiu de fer una comparació amb els millors fonaments possibles.

3 Disseny i Implementació

Pel nostre projecte, hem decidit dissenyar i implementar dos dels quatre tipus diferents d'estructures descrites anteriorment: un trie bàsic/naïve i un Radix Tree. Donada la significativa millora de rendiment teòrica d'un Radix Tree respecte a trie bàsic i el TST, hem decidit fer la seva implementació i estudiar de manera experimental el seu comportament. Cap recalcar, també, que la implementació d'un TST venia amb una dificultat afegida provinent de l'ordre d'inserció de les paraules, ja que, com s'ha descrit amb anterioritat, és molt important per una millora d'eficiència significativa que sigui una estructura d'arbre balancejada. En cas de fer servir una entrada amb les paraules en ordre alfabètic, com veurem en els apartats següents, el TST presentaria un clar desavantatge respecte el Radix Trie, i fins i tot, respecte la implementació Naïve. Això justifica la decisió de comparar activament el Radix Tree amb una base Naïve, quedant l'exploració de les capacitats dels TST com a possibilitat per a una investigació futura.

En aquest apartat es detallen les nostres decisions d'implementació de les estructures que utilitzarem per la fase experimental.

3.1 Trie Naïve

La primera implementació que hem realitzat ha sigut la del Naïve Trie. Hem creat una classe **NaïveTrie** i un struct **TrieNode** que conté un node arrel i diverses funcions per a la inserció, cerca de paraules i prefixos. Cada node del trie està representat per una estructura que conté un vector *children* de punters als seus fills (de mida 128 per cobrir l'alfabet ASCII), un vector *index* que indica totes les posicions on ha aparegut la paraula al text (útil per cert usos) i un indicador *end of word* per saber si el node marca el final de paraula. Hem intentat aplicar els bàsics d'un trie però fent-lo una mica ineficient per representar una implementació naïve.

També hem implementat la capacitat de buscar prefixos, utilitzant una funció que retorna totes les paraules en el subtree del node actual. Aquesta funció recursiva recorre tots els fills del node i afegeix les paraules completes a una llista de resultats. Aquestes implementacions no tenen com a objectiu ser les més correctes, sinó fer una implementació naïve que permeti comparar l'eficiència temporal i espacial amb el Radix Tree, una estructura més sofisticada. Tot i això, hem fet que la inserció i la búsqueda tinguin cost lineal $O(n)$ respecte el tamany n de la paraula a buscar/inserir.

3.2 Radix Tree

El RadixTree és l'estructura més compacta i eficient que hem implementat, dissenyada per maximitzar l'estalvi d'espai al comprimir les arrels comunes dels prefixos en etiquetes de longitud variable. La nostra implementació funciona com un Trie de Sufixos, indexant la totalitat dels sufixos del text d'entrada (com s'executa a la funció `init(text)`), cosa crucial per permetre la cerca de qualsevol fragment dins de la seqüència. El node intern (**RadixNode**) emmagatzema una *Label* (la subcadena comprimida que representa un camí lineal), punters als seus fills, i un vector *index* per a les posicions al text original.

La lògica d'inserció (**insert**) és l'operació més complexa, ja que requereix la gestió explícita de la compressió. Si una nova paraula a inserir només coincideix parcialment amb la *Label* d'un node fill existent, es produeix un mismatch i el node s'ha de dividir. Es crea un nou node intermedi amb la porció de la *Label* que coincideix (el prefix comú), i tant el node original com el nou sufix s'adjunten com a fills. Aquesta lògica de divisió assegura que l'arbre es mantingui en el seu format compacte i estalviï espai.

Pel que fa a la cerca de fragments (**search**), hem corregit la lògica per adaptar-la al seu ús com a Trie de Sufixos. La cerca no depèn del marcador `end_of_word`, ja que el fragment (com un patró de DNA) pot estar al mig d'un sufix més llarg. El fragment es considera trobat si es pot recórrer la totalitat del fragment navegant amb èxit a través de les *Labels* de l'arbre. Per obtenir les ocurrencies, la funció executa una recol·lecció recursiva de totes les posicions (**index**) a partir del node on finalitza el fragment. Aquesta estructura ofereix la màxima eficiència teòrica amb operacions de cerca i inserció realitzades en un temps $O(k)$, on k és la longitud del fragment o paraula a cercar/inserir. El cost és independent del nombre total de paraules emmagatzemades al Trie.

4 Avaluació Experimental

Per tal de fer una experimentació fiable, hem fet servir datasets de diferents característiques. A més, hem implementat diferents modes d'indexació per les paraules dels nostres datasets, de manera que hem analitzat els canvis que succeeixen quan canviem aquest paràmetre. En aquest cas, el primer mode (o mode 0) d'indexació es basa en la indexació per posició de paraula al text. Molt senzillament ha quedat implementat com un contador incremental a mesura que s'indexen totes les paraules del nostre dataset. El segon mode es tracta d'una indexació per línia de text, útil per sets de dades amb característiques concretes, com pot ser, per exemple, `wikipedia_titles.gz` la llista de noms d'articles de la Wikipedia. En aquest cas, veiem que ens seria poc útil una indexació per posició de paraula, si treballem amb l'objectiu de trobar els articles que continguin el que estem buscant. Un exemple del cas contrari, però, seria l'arxiu `words_alpha.txt` que conté totes les paraules d'un diccionari en anglès, una per línia. Aquí es fa fàcil veure que la indexació serà igual en un mode o en l'altre. Un cas particular de les entrades que hem fet servir per fer els nostres experiments és el cas de `dna_genome.txt`, un arxiu que conté aproximadament 12.0 MB d'informació d'un genoma real humà. Aquest dataset difereix de la resta d'exemples que hem fet servir en que es una única string continua, sense espais ni marcadors pel final de paraula.

Durant la nostra experimentació aquest arxiu ha estat tractat amb els canvis de línia com a separació entre paraules. Una possible implementació per la gestió d'aquest tipus d'arxiu seria la indexació de tots els sufixos existents de la string, guardant així tots els possibles subconjunts de la cadena. No ha sigut possible realitzar aquest enfoc per temes de recursos, ja que la mida dels arbres era massa gran per les màquines que teníem disponibles i els sistemes que hem fet servir. Tot i això, hem tingut en compte el conjunt de dades per les nostres experimentacions, per poder veure com es comportava respecte a altres entrades.

L'entorn d'experimentació ha sigut un PC amb processador IntelCore i7, 16GB de RAM i una targeta gràfica NVIDIA RTX 3080ti. El sistema operatiu és Linux Arch.

Les mètriques que hem decidit utilitzar per realitzar l'experimentació han sigut: memòria total ocupada pels dos tipus de tries un cop fetes totes les insercions, els nodes totals, que seran proporcional a la memòria ocupada, profunditat màxima i mitjana de cada estructura i temps de cerca per unes determinades paraules. Per cada fitxer que hem inserit als arbres lexicogràfics hem generat una sèrie de datasets per tal de realitzar les cerques de paraules aleatòries. Tots els datasets generats consisteixen d'un 30% de les paraules totals del fitxer, més un 10% de paraules que no apareixen als textos, per tal de poder estudiar també el comportament de les estructures quan la cerca de paraules resulta fallida.

4.1 Resultats

Els resultats de la nostra investigació han estat força consistents amb les hipòtesis teòriques que ens havíem suposat. El Radix Tree ha demostrat ser molt superior en termes de memòria, encara que no presenta unes millores significativament altes en temps de cerca ni inserció.

4.2 Eficiència Espacial

El Radix Tree demostra una superioritat clara en termes d'ús de memòria. En datasets com `moby_dick`, el Radix Tree ocupa aproximadament 4.96 MB front als 51.86 MB del Naive Trie (una reducció del 90.4%). Aquesta diferència s'accentua encara més en datasets amb moltes paraules úniques: per `words_alpha`, el Radix utilitza 83.61 MB mentre que el Naive requereix 1.08 GB, obtenint una compressió del 92.3%. En casos extrems com `dna_genome`, la millora arriba al 99.3% (75.35 MB vs 10.36 GB). Aquestes dades confirmen que la compressió de prefixos comuns del Radix Tree és altament efectiva en entorns amb recursos limitats.

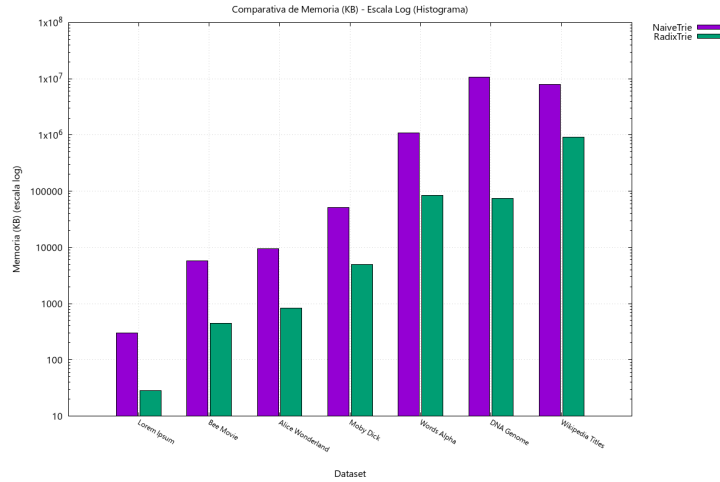


Figura 4: Representació en escala logarítmica de l'ús de memòria en KB per cada dataset i tipus de trie

4.3 Eficiència Temporal

Els temps d'inserció mostren resultats mixtos. En datasets petits com `lorem_ipsum`, el Naive és més ràpid (0.81 ms vs 1.23 ms del Radix). No obstant això, en datasets grans com `dna_genome`, el Radix Tree presenta millores significatives en temps d'inserció (651 ms vs 2558 ms) i cerca (262 ms vs 541 ms), demostrant un millor rendiment asimptòtic. En el cas de `words_alpha`, el Radix és més eficient en inserció però lleugerament més lent en cerca, probablement degut al cost de navegació per les etiquetes comprimides.

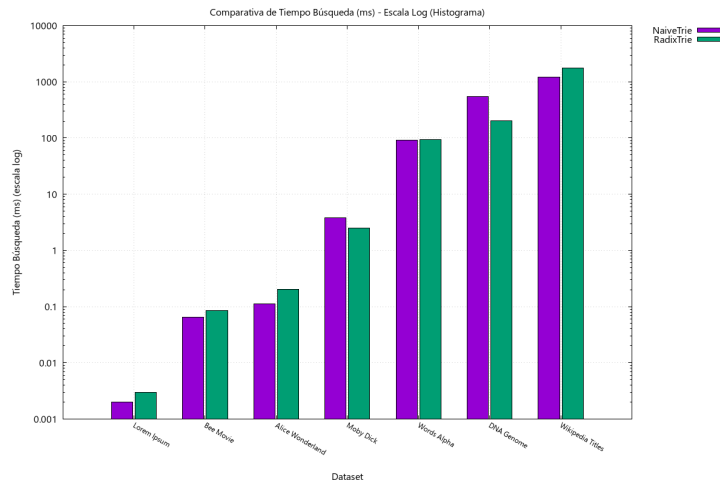


Figura 5: Representació en escala logarítmica del temps de cerca en ms per cada dataset i tipus de trie.

4.4 Nodes Visitats i Profunditat

El Radix Tree redueix significativament el nombre de nodes necessaris per representar el mateix conjunt de dades. Per exemple, a `moby_dick`, el Radix utilitza 45.258 nodes front als 63.712 del Naive (una reducció del 29%). Aquesta compactació es tradueix en arbres menys profunds i, en general, en menor nombre de nodes visitats durant les cerques.

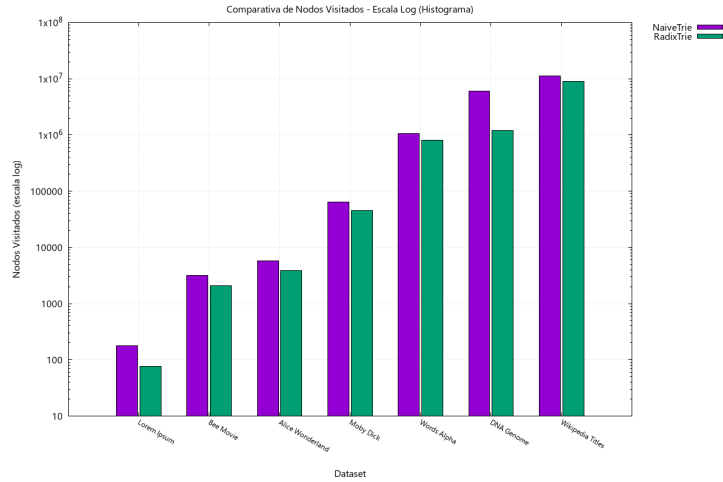


Figura 6: Representació en escala logarítmica dels nodes visitats per cada dataset i tipus de trie

Quant la profunditat, també observem una mediana més baixa al Radix Tree respecte al Naive Trie, com era d'esperar. Podem observar una diferència molt notable al dataset **human.genome**, per exemple, que té un nombre fixe de lletres per paraula i que per tant, té una profunditat màxima i mitjana de 50 nodes (la mida d'una cadena). Al radix tree, la compressió de nodes fa que es redueixi aproximadament un 80% de la seva mida original. A la figura següent es representen aquestes dades.

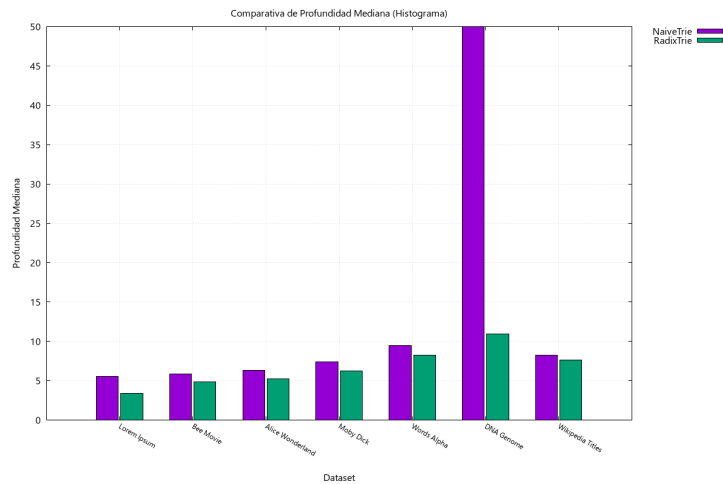


Figura 7: Representació de la profunditat mitjana de les estructures per cada dataset i tipus de trie.

5 Conclusions

Aquest projecte ha permès analitzar experimentalment el comportament de dues estructures de dades fonamentals per a l'emmagatzematge eficient de paraules: el Trie Naive i el Radix Tree. Els resultats obtinguts confirmen les hipòtesis teòriques plantejades a l'inici i permeten extreure les següents conclusions principals:

5.1 Aplicabilitat i Limitacions

Els resultats suggereixen que el Radix Tree és especialment adequat per aplicacions on la memòria és un recurs crític i es treballa amb grans volums de text amb prefixos comuns (diccionaris, autocompletat, indexació de genomes). El Naive Trie, tot i ser més simple d'implementar, només és viable per datasets molt petits o quan la simplicitat de codi és prioritària.

Una limitació observada és que en datasets amb molt poca redundància de prefixos (com `wikipedia_titles`), l'overhead de gestió dels nodes comprimits pot fer que el Radix sigui menys eficient temporalment (15.26 s vs 8.62 s en inserció), tot i mantenir una millora substancial en memòria (88.6% de reducció).

5.2 Línies Futures

Com a possibles extensions del projecte, es proposen: (1) la implementació d'un Ternary Search Tree balancejat per avaluar-ne el rendiment amb datasets aleatoris, (2) l'estudi dels Patricia Trees per veure si aporten millores addicionals, (3) l'optimització del Radix Tree per millorar els temps de cerca en datasets amb poques coincidències de prefixos, i (4) l'avaluació d'aquestes estructures en operacions d'autocompletat en temps real amb datasets dinàmics.

En resum, aquest treball ha demostrat empíricament la superioritat del Radix Tree en eficiència espacial i ha quantificat els trade-offs temporals en diferents escenaris, proporcionant una base sòlida per a la selecció de l'estructura adequada segons els requisits de cada aplicació.

Part II. Informe d'Autoaprenentatge

6 Descripció i valoració del procés d'autoaprenentatge

Ara que ja hem acabat el projecte, descriurem en aquest apartat la nostra metodologia i el procés d'autoaprenentatge que hem seguit al llarg del desenvolupament.

6.1 Metodologia

Hem estructurat el desenvolupament del projecte en tres fases diferents:

1. **Fase d'investigació:** El primer pas va ser entendre bé els diferents tipus d'arbres lexicogràfics. Vam començar estudiant els tries bàsics i les seves variants (TST, Radix i Patricia), analitzant els seus avantatges i inconvenients teòrics. Amb aquesta informació vam decidir implementar el radix i com hauriem de fer-ho.
2. **Fase d'implementació:** Vam decidir començar amb una implementació bàsica d'un trie per implementar la resta a partir d'allò. Després vam crear el naive trie per establir una base de comparació amb el radix. Això ens va permetre entendre els conceptes fonamentals abans de la implementació més complicada del Radix Tree. La decisió de no implementar el TST va ser a causa de la seva variació d'estructura i rendiment depenent de l'ordre d'inserció de les paraules, i que després de la investigació vam veure que el Radix semblava molt més interessant d'implementar.
3. **Fase d'experimentació:** Finalment, vam dissenyar i executar proves per comparar el rendiment de les nostres implementacions, centrant-nos en mesures com l'ús de memòria i el temps d'execució de les operacions bàsiques amb els diferents inputs els quals ja han sigut descrits al treball.

Per la divisió del treball, la fase d'investigació va ser feta de manera conjunta, amb tot el equip investigant i discutint els diversos tipus d'estructures i posteriorment, com descriure-les en el informe. Per la implementació, vam dividir el treball de tal manera que cada membre hagués de ocupar-se d'una estructura, menys del Radix, que per la seva complexitat afegida vam pensar que era millor que fos treballat per dos membres. La fase final d'experimentació ha sigut liderada pels membres que havien tingut menys treball durant les altres parts, tot i que els disseny dels mains va requerir un esforç conjunt.

6.2 Valoració del procés d'autoaprenentatge

La realització d'aquest projecte ha estat un procés molt interessant que ha permet ampliar els nostres coneixements d'aquest tipus d'estructures. El principal aprenentatge del pojecte ha sigut la validació pràctica de la idea que com més complex és l'algorisme, més eficient pot ser en el consum de recursos. Hem pogut veure com una estructura més complicada com el Radix Tree podia ser molt més eficient i escalable a llarg termini gràcies a la compactació estructural. També hem après a valorar més l'eficiència espacial, que tot i que no és tan relevant en els projectes que hem fet anteriorment, és un concepte molt important a l'hora de dissenyar estructures de dades.

El repte més gran va ser, sens dubte, la implementació de la lògica d'inserció del Radix Tree, especialment la gestió de la divisió de nodes i la redirecció de punters. Aquest procés va exigir entendre al detall en el maneig de subcadenes i ha consolidat la nostra habilitat per resoldre problemes algorísmics de gran complexitat.

També hem millorat en l'anàlisi crític de dades per poder extreure conclusions en termes d'eficiència. En general, considerem que els objectius inicials del projecte s'han assolit i que ens han proporcionat una base sòlida per a la comprensió i aplicació de les estructures de dades prefixades.

Referències

- [1] R. Engibaryan, *The Ternary Search Tree Data Structure*, Baeldung on Computer Science, March 17, 2023. Available: <https://www.baeldung.com/cs/ternary-search-tree>.
- [2] M. Maabar, *Trie Data Structure*, CVR Bioinformatics, November 17, 2014. Available: <https://bioinformatics.cvr.ac.uk/trie-data-structure/>.
- [3] A. Sharma and T. Goyal, *Real World Applications of Tries* [PDF], Academia.edu. Available: https://www.academia.edu/download/104436863/Research_Paper.pdf.
- [4] D. P. Mehta and S. Sahni, *Handbook of Data Structures and Applications*, Chap. 28: Tries, Chapman and Hall/CRC, 2004. Available: https://dpvipracollege.ac.in/wp-content/uploads/2023/01/Handbook_of_Data_Structures_and_Applications_Dinesh_P_Mehta.pdf.
- [5] E. Fredkin, “Trie memory” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, Sept. 1960. Available: <https://doi.org/10.1145/367390.367400>.
- [6] D. R. Morrison, “PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric,” *Journal of the ACM*, vol. 15, no. 4, pp. 514–534, Oct. 1968. Available: <https://doi.org/10.1145/321479.321481>.
- [7] K. Maly, “Compressed tries,” *Communications of the ACM*, vol. 19, no. 7, pp. 409–415, July 1976. Available: <https://doi.org/10.1145/359545.359548>.
- [8] J. Aoe, “An efficient digital search algorithm by using a double-array structure,” *IEEE Transactions on Software Engineering*, vol. 15, no. 9, pp. 1066–1077, Sept. 1989. Available: <https://doi.org/10.1109/32.31365>.
- [9] S. Nilsson and G. Karlsson, “IP-address lookup using LC-tries,” *IEEE Journal on Selected Areas in Communications*, vol. 17, no. 6, pp. 1083–1092, June 1999. Available: <https://doi.org/10.1109/49.772439>.
- [10] V. Leis, A. Kemper, and T. Neumann, “The adaptive radix tree: ARTful indexing for main-memory databases,” in *Proc. IEEE 29th Int. Conf. Data Engineering (ICDE)*, Brisbane, Australia, 2013, pp. 38–49. Available: <https://db.in.tum.de/~leis/papers/ART.pdf>.
- [11] H. Asai and Y. Ohara, “Poptrie: A compressed trie with population count for fast and scalable software IP routing table lookup,” in *Proc. ACM SIGCOMM Computer Communication Review*, vol. 45, no. 4, pp. 57–70, Aug. 2015. Available: <https://conferences.sigcomm.org/sigcomm/2015/pdf/papers/p57.pdf>.
- [12] A. Boffa, P. Ferragina, and G. Vinciguerra, “CoCo-trie: Data-aware compression and indexing of strings,” *Information Systems*, vol. 119, art. 102281, Jan. 2024. Available: <https://doi.org/10.1016/j.is.2023.102281>.

Apèndixs

En aquest apartat hem inclòs diverses representacions gràfiques útils per interpretar les dades.

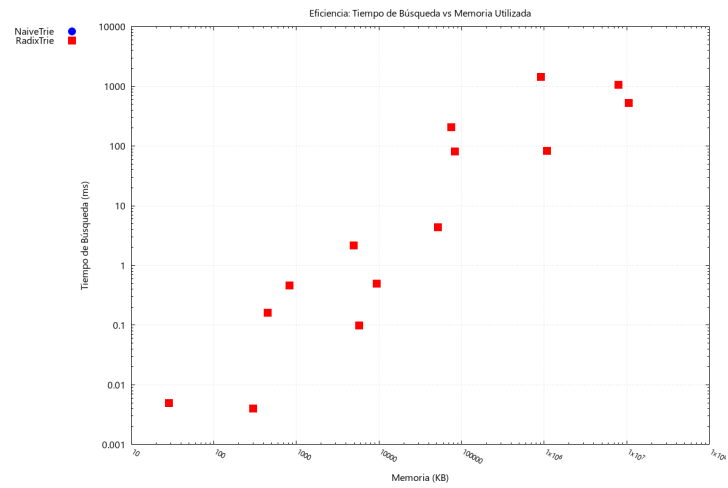


Figura 8: Representació gràfica de la eficiència de temps segons l'ús de memòria de cada estructura

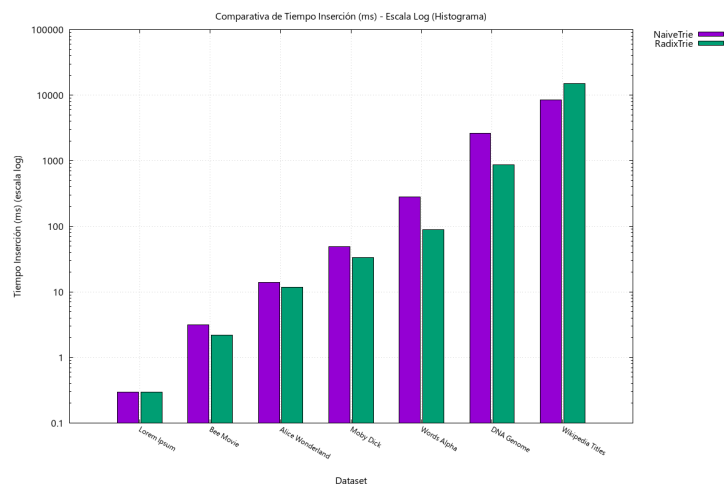


Figura 9: Representació en escala logarítmica del temps d'inserció per estructura i dataset

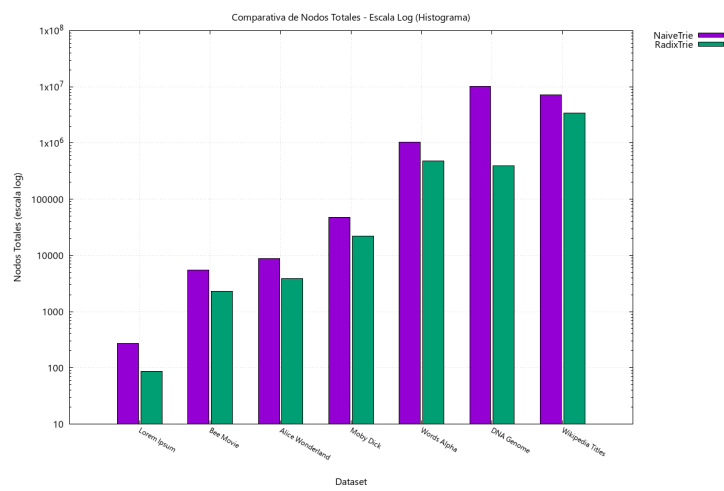


Figura 10: Representació gràfica dels Nodes Totals ocupats per estructura i dataset

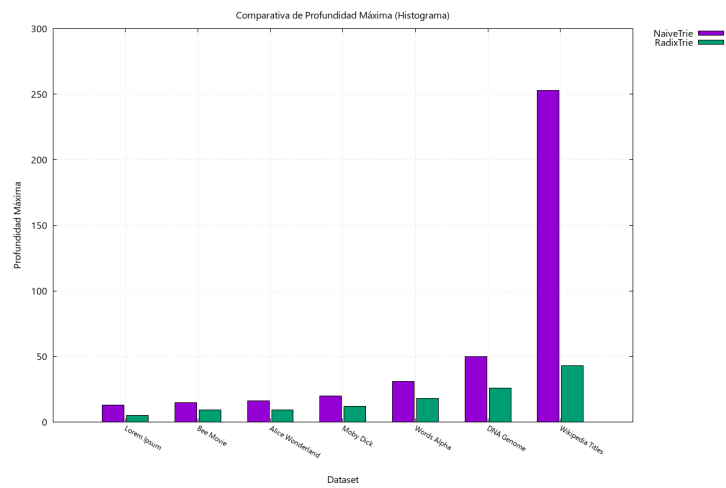


Figura 11: Representació gràfica de la profunditat màxima per estructura i dataset